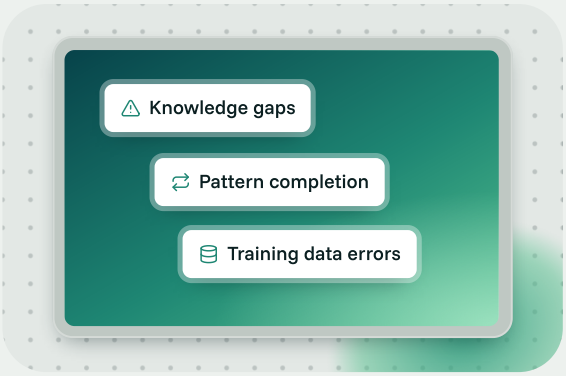
Last updated August 2026

Hallucinations

Why language models make things up, what causes confident but wrong output, and the techniques that reduce it.

12 min read · Beginner-friendly with advanced sections



Pascal Academy
AI fundamentals · Part 7 of 10

What Is Hallucination?

In the first guide, we established that a language model is a probability engine. It predicts the next token based on patterns learned from training data. The model does not know whether a statement is true. It knows whether the statement looks like text it has seen before. These two things are different, and the gap between them is where hallucination lives.

A hallucination is a statement produced by a language model that sounds plausible and specific but is factually wrong. The model might cite a research paper that does not exist, attribute a quote to the wrong person, invent a historical event, or describe an API method that was never part of the library. The output reads fluently because the model is good at producing text that looks like real text. The problem is that looking real and being real are not the same thing.

The term hallucination is borrowed from psychology, where it refers to perceiving something that is not there. In the LLM context, the model generates information that has no grounding in reality or in its training data. The model is not lying. Lying requires intent, and a language model has no intent. It is doing exactly what it was trained to do, which is to produce the most likely next token given the context. Sometimes the most likely token sequence is factually incorrect.

Key principle

Hallucination is a feature of how language models work, not a bug. The same mechanism that lets a model write fluent prose also lets it produce fluent fiction. The model cannot tell the difference between generating a correct fact and generating a plausible sentence. Reducing hallucination means giving the model external grounding so it does not have to rely on its own internal probabilities.

Why Hallucination Happens

To understand why models hallucinate, it helps to revisit how they generate text. A language model is trained to predict the next token in a sequence. During training, the model sees billions of sentences and learns the statistical patterns of language. It learns that the word Paris frequently appears near the word France, that the word CEO appears near company names, and that academic citations follow a specific format.

When you ask the model a question, it generates a response token by token. Each token is the one the model considers most likely given the context so far. If the model has strong training signal for the question, the likely tokens happen to also be the correct ones. If the training signal is weak or conflicting, the model still generates the most likely token, but that token may not correspond to a real fact.

Knowledge Gaps

The model's training data has gaps. It does not contain every fact about every topic. When you ask about something the model has rarely or never seen during training, it has weak signal to work from. The model still produces output because that is what it does, but the output is based on patterns from adjacent topics rather than on actual knowledge of the subject.

Pattern Completion

Language models are pattern completers. If the beginning of a sentence looks like a known pattern, the model completes it with the most likely continuation. Consider the pattern of an academic citation. A model that has seen millions of citations knows the format well. If you ask it for a paper on a specific

topic, it can produce a citation that looks perfect, with real-sounding author names, a plausible journal name, and a reasonable year. The citation is a pattern completion, not a retrieval of a real paper. The model has no mechanism to check whether the paper actually exists.

Training Data Errors

The training data itself contains errors. The public internet has misinformation, outdated information, contradictory claims, and plain mistakes. A model trained on this data learns these errors along with correct information. When it generates text, it can reproduce the errors it learned, and it has no way to distinguish between what it learned correctly and what it learned incorrectly.

Over-Confidence

Language models are trained to be helpful and informative. This training can push them toward producing an answer even when they do not have enough information. A model that says I do not know in response to every uncertain question is less useful than one that attempts an answer. The trade-off is that attempting answers to questions the model cannot answer leads to hallucination. Alignment training, like RLHF, encourages models to be helpful, which can inadvertently encourage confident-sounding answers even when the model should express uncertainty.

Types of Hallucination

Hallucinations come in several forms. Understanding the type helps in choosing the right mitigation.

Pascal Academy

Pascal Academy's LLM evaluation course covers hallucination detection, factuality benchmarking, and production monitoring. Reach out to learn about cohort programmes for your engineering team.

Ask about team training

Type	Description and Example
Factuality hallucination	The model states a fact that is demonstrably wrong. For example, claiming that Marie Curie won the Nobel Prize in Mathematics, a prize that does not exist. The statement is fluent and specific but has no basis in reality.
Fabricated source	The model invents a citation, reference, or link. For example, generating a paper title like Smith, J. (2023). Effects of Temperature on LLM Performance. Journal of AI Research, 45(3), 112-128. The format is correct but the paper does not exist.
Attribute error	The model assigns an action or quote to the wrong entity. For example, attributing a quote by Einstein to Newton, or saying a company was founded by the wrong person. Both entities are real, but the relationship between them is fabricated.
Contextual hallucination	The model contradicts information it was given in the prompt or in retrieved documents. For example, a RAG system retrieves a document stating the refund window is 30 days, but the model says 60 days. The correct information was available but the model ignored or overrode it.
Numerical hallucination	The model produces wrong numbers, dates, or statistics. For example, stating a population of 12 million for a city of 8 million, or giving a product price that does not match the catalogue.

What Makes Hallucination Worse

Several factors increase the likelihood and severity of hallucination. Being aware of these helps in designing systems that minimise the risk.

High Temperature

Temperature controls how random the model's token selection is. A high temperature means the model picks from a wider range of candidate tokens, including less likely ones. This increases creativity, which is useful for brainstorming or creative writing, but it also increases the chance that the model picks a token that leads away from a factual answer. For factual tasks, a low temperature reduces hallucination by keeping the model close to its highest-probability, and usually most accurate, tokens.

Ambiguous or Leading Prompts

The way a question is phrased affects the model's output. A prompt that assumes a false premise can lead the model to produce elaborate answers built on that false premise. If you ask a model to explain why Abraham Lincoln resigned from the presidency, the model may comply with an explanation even though Lincoln never resigned. The model follows the prompt's framing rather than correcting the assumption.

Niche or Obscure Topics

Hallucination is more common on topics that appear infrequently in training data. A model can answer common questions about world capitals or basic physics with high accuracy because these topics are well-represented in training data. Ask the same model about a specific provision in an obscure regulation, and it may hallucinate because it has very little relevant training signal.

Long Context with Many Entities

When the context contains many entities, dates, and relationships, the model can conflate details. As covered in the context windows guide, the lost-in-the-middle effect means the model pays less attention to information in the middle of long inputs. A model processing a 50-page document may correctly recall facts from the first and last pages but fabricate details from page 27.

Go deeper

The confidence-accuracy gap

Optional

A language model's output does not carry a reliable confidence score. When the model says something, it sounds equally confident whether the statement is a well-established fact or a complete fabrication. This is because the model's training optimises for fluent text generation, not for calibrated uncertainty. Some research approaches attempt to extract confidence estimates from the model's token probabilities, asking the model to generate multiple answers and measuring agreement, or training the model to express uncertainty in words. These approaches help but do not fully solve the problem. The model can be confidently wrong, and no amount of probability extraction can tell you whether a specific factual claim is correct without external verification.

Reducing Hallucination

No single technique eliminates hallucination. Production systems use a combination of approaches, each addressing a different cause.

Pascal Academy

Pascal Academy's RAG engineering and prompt engineering courses cover hallucination mitigation in depth, including grounding patterns, output validation, and evaluation frameworks. Reach out to learn about team training options.

Ask about team training

Technique	How It Reduces Hallucination
RAG	Retrieves external documents so the model extracts answers from text rather than relying on training-data memory.
Prompt engineering	Instructs the model to ground answers in provided context and to say it does not know when information is missing.
Knowledge graphs (GraphRAG)	Provides structured, verified relationships for the model to reason over, with traceable sources.
Low temperature	Keeps token selection close to the model's highest-probability predictions, reducing random deviations into fabrication.
Output validation	Checks generated output against external sources and blocks or flags responses containing unverified claims.
Chain-of-thought	Asks the model to show reasoning steps, making errors visible and sometimes triggering self-correction.
Multiple sampling	Generates several answers and compares them. High agreement suggests confidence. Disagreement signals uncertainty.

Retrieval-Augmented Generation (RAG)

RAG is the single most effective tool against hallucination. Instead of asking the model to answer from its training data, you retrieve relevant documents and include them in the prompt. The model then extracts and synthesises information from those documents rather than generating from memory. As covered in the RAG guide, this changes the task from recall to extraction, and extraction is something language models do well.

RAG does not eliminate hallucination entirely. The model can still misread a retrieved document or invent information when the document does not contain the answer. But the risk is lower because the model has concrete text to anchor its response to, and you can instruct it to say it does not know when the retrieved documents lack the answer.

Prompt Engineering

The way you phrase instructions affects hallucination. Telling the model to answer only based on the provided documents, and to say it does not know when the documents do not contain the answer, reduces fabricated responses. Adding an instruction like if you are not sure, say you do not know rather than guessing gives the model permission to express uncertainty.

Asking the model to show its reasoning, through chain-of-thought prompting, can also help. When the model writes out its reasoning steps, errors become visible and the model sometimes corrects itself mid-generation. This is more effective for reasoning tasks than for factual recall.

Knowledge Graphs and Grounding

As covered in the knowledge graphs guide, GraphRAG provides structured relationship data that the model can reason over. When the model has access to a knowledge graph that encodes verified facts and their sources, it can ground its answers in those relationships rather than in its training data. This is particularly useful for questions that require multi-hop reasoning across entities, where the model's internal knowledge is likely to be incomplete or wrong.

Output Validation and Guardrails

Post-generation validation checks the model's output against external sources before presenting it to the user. A validation layer can check whether cited papers exist, whether numbers match a database, and whether claims are consistent with known facts. When the validation layer detects a problem, it can block the response, ask the model to regenerate, or flag the response for human review.

Guardrails are rules that constrain the model's behaviour. They can prevent the model from generating content on certain topics, from making specific types of claims, or from producing output that violates safety guidelines. Guardrails do not prevent hallucination directly, but they limit the damage by catching problematic output before it reaches the user.

Measuring Hallucination

You cannot reduce what you cannot measure. Evaluating hallucination requires a test set of questions with known correct answers, and a method for checking whether the model's responses match those answers.

Human Evaluation

The gold standard is human review. A domain expert reads the model's responses and checks each factual claim against a trusted source. This is accurate but expensive and slow, making it impractical for continuous evaluation at scale.

LLM-as-a-Judge

A common approach is to use a separate, stronger language model to evaluate the output of the model being tested. The judge model is given the question, the model's response, and the ground truth answer, and asked to determine whether the response is factually correct. This scales better than human evaluation and correlates reasonably well with human judgments, but it introduces its own biases. The judge model can hallucinate when evaluating hallucination.

Factuality Benchmarks

Several benchmarks exist for measuring factuality. These include datasets of questions with verified answers, where the model's responses are automatically checked for correctness. Examples include TruthfulQA, which tests whether models resist common misconceptions, and FActScore, which breaks down responses into individual atomic facts and checks each one. These benchmarks give a rough sense of a model's tendency to hallucinate, but they cover a narrow range of topics and may not reflect performance on your specific domain.

Go deeper

Why evaluation is hard

Optional

Hallucination evaluation is hard because correctness is not always binary. A response can be partially correct, correct in spirit but wrong in detail, or correct on the main point but wrong on a supporting claim. Automated evaluators struggle with these nuances. A response that says the refund window is 30 days is wrong if the actual window is 14 days, but an LLM-as-a-judge might rate it as close enough. The granularity of evaluation matters. Checking whether the response answers the question at all is easier than checking every factual claim within it. Production systems often combine automated evaluation for scale with periodic human review for calibration.

Hallucination in Practice

The practical impact of hallucination depends on the application. For a creative writing tool, hallucination is part of the feature. For a medical chatbot, it can be dangerous. For a legal research assistant, it can lead to professional liability.

High-Stakes Applications

In healthcare, finance, and legal applications, hallucination carries real consequences. A model that invents a drug interaction, fabricates a regulatory requirement, or cites a non-existent legal precedent can cause harm that no disclaimer fully addresses. These applications require the strongest mitigations, including RAG with verified sources, output validation against authoritative databases, human-in-the-loop review for critical decisions, and clear disclaimers about the limitations of AI-generated content.

Low-Stakes Applications

For content generation, brainstorming, and casual Q and A, hallucination is less critical. A model that suggests a creative plot twist that does not make historical sense is not going to harm anyone. The

appropriate response is to calibrate the level of mitigation to the stakes of the application, rather than applying the same heavy-handed safeguards everywhere.

User Trust

Hallucination erodes user trust. When a user catches a model stating something confidently that is wrong, they begin to distrust all of its output, including the parts that are correct. This is particularly damaging for enterprise applications where the model is supposed to be a reliable assistant. Transparency about limitations, citation of sources, and clear marking of uncertainty help maintain trust even when the model occasionally makes mistakes.

The Future of Hallucination

Hallucination has been a known problem since the first large language models were deployed, and it remains unsolved in the general case. Progress has been made through RAG, grounding techniques, and better training methods, but no current model can guarantee factual accuracy on arbitrary questions.

Reasoning models, covered in the next guide, offer a partial path forward. By spending more compute on thinking through a problem step by step, these models can catch some of their own errors before producing output. This helps with reasoning and mathematical errors but does not solve the fundamental problem of factual gaps in training data.

Research is active on several fronts. Training methods that teach models to express calibrated uncertainty, architectures that separate factual recall from text generation, and retrieval systems that provide real-time fact-checking are all being explored. The most promising direction may be combining language models with external knowledge systems, like knowledge graphs and verified databases, so the model generates text and grounds it in structured, verifiable facts simultaneously.

Summary

Hallucination is the production of confident, fluent, and factually wrong statements by a language model. It happens because models generate text based on statistical patterns, not on verified knowledge. The model cannot distinguish between a plausible sentence and a true one. Hallucination is worse on obscure topics, with high temperature, and when prompts contain false premises. The main mitigations are RAG to ground answers in retrieved documents, prompt engineering to encourage uncertainty, knowledge graphs for structured grounding, low temperature for factual tasks, and output validation to catch errors before they reach the user. No single technique eliminates hallucination, and production systems combine multiple approaches. Measuring hallucination requires evaluation sets with known correct answers, and methods range from human review to automated factuality benchmarks. The stakes determine how much mitigation is needed, with healthcare, legal, and financial applications requiring the strongest safeguards.

About this guide

This guide is part of Pascal Academy's AI Fundamentals series, covering LLMs, prompt engineering, RAG, agents, context windows, knowledge graphs, hallucinations, reasoning models, embeddings, and MCP. The full series and hands-on courses are available at Pascal Academy. For teams looking to upskill, we offer custom cohort programmes tailored to your stack and use cases.

Frequently asked questions

What are AI hallucinations in finance?

Hallucinations are outputs where an AI model generates information that sounds plausible but is factually incorrect. In finance, this could mean a model inventing a revenue figure,

fabricating a quote from an earnings call, or citing a filing that does not contain the claimed data. Hallucinations are the primary reason verification is essential.

Why do AI models hallucinate?

Models hallucinate because they generate text by predicting the next likely token, not by looking up facts. When they lack specific information, they fill gaps with plausible-sounding content. Hallucinations are more common when prompts are vague, when the model lacks context, or when the question involves specific numbers or dates.

How does Pascal reduce hallucinations in financial research?

Pascal reduces hallucinations by grounding every output in source documents through retrieval-augmented generation. Instead of generating from training data alone, Pascal's agents pull from actual filings and your internal research. Every claim includes a citation tracing back to the source line, so you can verify accuracy before relying on the output.

How can you detect and prevent AI hallucinations?

Detect hallucinations by checking every number against the source, tracing every claim to its origin, and verifying the reasoning. Prevent them by providing specific context in your prompts, using RAG to ground outputs in real documents, and building verification checkpoints into your workflow. Never present an AI output to a stakeholder without independent review.